# Fast Sector Decomposition from Black-Box Symanzik Polynomials

## 1 The example triangle

We consider the scalar one-loop triangle with one incoming off-shell momentum and two outgoing massless momenta,

$$p_0 = p_1 + p_2, \qquad p_1^2 = p_2^2 = 0, \qquad p_0^2 = s = m_0^2. \tag{1}$$

All three internal propagators are taken to have the same mass $m$. A convenient routing is

$$D_0 = \ell^2 - m^2 + \mathrm{i}0, \tag{2}$$
$$D_1 = (\ell + p_1)^2 - m^2 + \mathrm{i}0, \tag{3}$$
$$D_2 = (\ell - p_2)^2 - m^2 + \mathrm{i}0. \tag{4}$$

The scalar integral is

$$C_0(s; m^2) = \int \frac{\mathrm{d}^D \ell}{\mathrm{i}\pi^{D/2}} \frac{1}{D_0 D_1 D_2}. \tag{5}$$

With Feynman parameters $x_i \geq 0$, the projective representation is

$$C_0(s; m^2) = -\Gamma(\alpha) \int_0^\infty \mathrm{d}x_0\, \mathrm{d}x_1\, \mathrm{d}x_2\, \delta(1 - x_0 - x_1 - x_2)\, \frac{\mathcal{U}^{3-D}}{\mathcal{F}^\alpha}, \qquad \alpha = 3 - \frac{D}{2}. \tag{6}$$

For this routing,

$$\mathcal{U} = x_0 + x_1 + x_2, \tag{7}$$

and

$$\mathcal{F} = m^2 \mathcal{U}^2 - s\, x_1 x_2 - \mathrm{i}0. \tag{8}$$

Solving the simplex constraint by

$$x_0 = 1 - x_1 - x_2 \tag{9}$$

gives

$$0 \leq x_1 \leq 1, \qquad 0 \leq x_2 \leq 1 - x_1, \tag{10}$$

and therefore

$$\mathcal{U} = 1, \qquad \mathcal{F} = m^2 - s\, x_1 x_2 - \mathrm{i}0. \tag{11}$$

For $D = 4 - 2\epsilon$, one has $\alpha = 1 + \epsilon$, and

$$C_0(s; m^2) = -\Gamma(1 + \epsilon) \int_0^1 \mathrm{d}x_1 \int_0^{1-x_1} \mathrm{d}x_2\, \left(m^2 - s\, x_1 x_2 - \mathrm{i}0\right)^{-1-\epsilon}. \tag{12}$$

Throughout the rest of the note, sector integrands $I_s$ do not include the global prefactor $-\Gamma(\alpha)$. Thus an exact decomposition has the form

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_s \int_{\mathcal{C}_s} \mathrm{d}y_s\, I_s(y_s). \tag{13}$$

### 1.1 Endpoint sectors

The primary endpoint decomposition splits the simplex into two sectors.

**Sector $S_1$, $x_1 \geq x_2$.**

$$X_{S_1}(t,z) = (x_0, x_1, x_2) = \left(1-t,\, \frac{t}{1+z},\, \frac{tz}{1+z}\right), \qquad 0 \leq t, z \leq 1. \tag{14}$$

The Jacobian is

$$J_{S_1}(t,z) = \frac{t}{(1+z)^2}. \tag{15}$$

The pulled-back second Symanzik polynomial is

$$\mathcal{F}_{S_1}(t,z) = m^2 - s\,\frac{t^2 z}{(1+z)^2} - \mathrm{i}0. \tag{16}$$

In the massless endpoint case $m = 0$,

$$\mathcal{F}_{S_1}(t,z) = (-s - \mathrm{i}0)\,\frac{t^2 z}{(1+z)^2}. \tag{17}$$

The expected monomial is therefore

$$M_{S_1}(t,z) = t^2 z. \tag{18}$$

**Sector $S_2$, $x_2 \geq x_1$.**

$$X_{S_2}(t,z) = (x_0, x_1, x_2) = \left(1-t,\, \frac{tz}{1+z},\, \frac{t}{1+z}\right), \qquad 0 \leq t, z \leq 1. \tag{19}$$

The Jacobian is again

$$J_{S_2}(t,z) = \frac{t}{(1+z)^2}. \tag{20}$$

The pulled-back polynomial is

$$\mathcal{F}_{S_2}(t,z) = m^2 - s\,\frac{t^2 z}{(1+z)^2} - \mathrm{i}0. \tag{21}$$

In the massless endpoint case,

$$\mathcal{F}_{S_2}(t,z) = (-s - \mathrm{i}0)\,\frac{t^2 z}{(1+z)^2}, \qquad M_{S_2}(t,z) = t^2 z. \tag{22}$$

With these definitions, the two-sector representation of the original triangle is

$$C_0(s;m^2) = -\Gamma(\alpha) \sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, I_{S_a}(t,z), \tag{23}$$

$$I_{S_a}(t,z) = \frac{t}{(1+z)^2}\, \mathcal{U}(X_{S_a}(t,z))^{3-D}\, \mathcal{F}(X_{S_a}(t,z))^{-\alpha}. \tag{24}$$

Equivalently, after inserting the explicit polynomial,

$$C_0(s;m^2) = -\Gamma(\alpha) \sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, \frac{t}{(1+z)^2} \left(m^2 - s\,\frac{t^2 z}{(1+z)^2} - \mathrm{i}0\right)^{-\alpha}. \tag{25}$$

For $D = 4 - 2\epsilon$ and $m = 0$, this becomes

$$C_0(s;0) = -\Gamma(1+\epsilon) \sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, t^{-1-2\epsilon} z^{-1-\epsilon} G_{S_a}(t,z), \tag{26}$$

where

$$G_{S_a}(t,z) = (-s - \mathrm{i}0)^{-1-\epsilon} (1+z)^{2\epsilon}. \tag{27}$$

## 2 Direct approach

In the direct approach, each sector map $X_s$ is substituted explicitly into $\mathcal{U}$ and $\mathcal{F}$. If the pulled-back polynomial factorizes as

$$\mathcal{F}_s(y) = \mathcal{F}(X_s(y)) = M_s(y)\,\phi_s(y), \tag{28}$$

then the sector integrand is built as

$$I_s^{\mathrm{dir}}(y) = J_s(y)\,\mathcal{U}(X_s(y))^{3-D}\,M_s(y)^{-\alpha}\,\phi_s(y)^{-\alpha}. \tag{29}$$

The corresponding contribution to the original scalar integral is always understood as

$$C_0 = -\Gamma(\alpha)\sum_s \int_{\mathcal{C}_s} \mathrm{d}y_s\, I_s^{\mathrm{dir}}(y_s), \tag{30}$$

with the sector domains $\mathcal{C}_s$ specified explicitly below.

### 2.1 Endpoint sector $S_1$

For $m = 0$,

$$\mathcal{U}_{S_1}(t,z) = 1, \tag{31}$$

and

$$\mathcal{F}_{S_1}(t,z) = (-s - \mathrm{i}0)\,\frac{t^2 z}{(1+z)^2} = M_{S_1}(t,z)\,\phi_{S_1}(t,z), \tag{32}$$

with

$$M_{S_1}(t,z) = t^2 z, \qquad \phi_{S_1}(t,z) = \frac{-s-\mathrm{i}0}{(1+z)^2}. \tag{33}$$

Therefore

$$I_{S_1}^{\mathrm{dir}}(t,z) = \frac{t}{(1+z)^2}\,(t^2 z)^{-\alpha}\left(\frac{-s-\mathrm{i}0}{(1+z)^2}\right)^{-\alpha}. \tag{34}$$

For $D = 4-2\epsilon$, i.e. $\alpha = 1+\epsilon$,

$$I_{S_1}^{\mathrm{dir}}(t,z) = (-s-\mathrm{i}0)^{-1-\epsilon}\,t^{-1-2\epsilon}\,z^{-1-\epsilon}\,(1+z)^{2\epsilon}. \tag{35}$$

### 2.2 Endpoint sector $S_2$

By symmetry,

$$\mathcal{F}_{S_2}(t,z) = (-s-\mathrm{i}0)\,\frac{t^2 z}{(1+z)^2}, \qquad M_{S_2}(t,z) = t^2 z, \tag{36}$$

and hence

$$I_{S_2}^{\mathrm{dir}}(t,z) = (-s-\mathrm{i}0)^{-1-\epsilon}\,t^{-1-2\epsilon}\,z^{-1-\epsilon}\,(1+z)^{2\epsilon}. \tag{37}$$

The full massless triangle is therefore

$$\begin{aligned} C_0(s;0) &= -\Gamma(1+\epsilon)\sum_{a=1}^{2}\int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, I_{S_a}^{\mathrm{dir}}(t,z) \\ &= -2\Gamma(1+\epsilon)(-s-\mathrm{i}0)^{-1-\epsilon}\int_0^1 \mathrm{d}t\int_0^1 \mathrm{d}z\, t^{-1-2\epsilon} z^{-1-\epsilon}(1+z)^{2\epsilon}. \end{aligned} \tag{38}$$

## 3 Indirect approach

The indirect approach keeps the Symanzik polynomials closed. We assume only black-box evaluators

$$\widehat{\mathcal{U}}(x_0, x_1, x_2), \qquad \widehat{\mathcal{F}}(x_0, x_1, x_2), \tag{39}$$

together with their partial derivatives with respect to the original parameters $x_i$. In practice, these derivative jets can be computed efficiently with dual numbers.

For a sector map $X_s$, define the pullbacks

$$U_s(y) = \widehat{\mathcal{U}}(X_s(y)), \qquad F_s(y) = \widehat{\mathcal{F}}(X_s(y)). \tag{40}$$

Derivatives with respect to sector variables are obtained by chain rules. For example,

$$\partial_a F_s(y) = \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \, \frac{\partial X_{s,i}}{\partial y_a}(y), \tag{41}$$

and

$$\begin{aligned} \partial_a \partial_b F_s(y) = &\sum_{i,j} \frac{\partial^2 \widehat{\mathcal{F}}}{\partial x_i \partial x_j}(X_s(y)) \, \frac{\partial X_{s,i}}{\partial y_a}(y) \, \frac{\partial X_{s,j}}{\partial y_b}(y) \\ &+ \sum_i \frac{\partial \widehat{\mathcal{F}}}{\partial x_i}(X_s(y)) \, \frac{\partial^2 X_{s,i}}{\partial y_a \partial y_b}(y). \end{aligned} \tag{42}$$

Higher derivatives are obtained similarly, or more directly by propagating dual-number jets through the known sector map and then through the black-box polynomial evaluator.

The object to construct is the regular coefficient $\phi_s$ defined by

$$F_s(y) = M_s(y) \, \phi_s(y), \tag{43}$$

where $M_s$ is the expected monomial for that sector. Once $\phi_s$ is available, the sector integrand is

$$I_s^{\mathrm{ind}}(y) = J_s(y) \, U_s(y)^{3-D} \, M_s(y)^{-\alpha} \, \phi_s(y)^{-\alpha}. \tag{44}$$

At interior points this is pointwise identical to direct substitution, since

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)} \tag{45}$$

there. For the endpoint decomposition before any monomial extraction, the black-box form of the exact triangle is

$$C_0(s; m^2) = -\Gamma(\alpha) \sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z \, \frac{t}{(1+z)^2} \widehat{\mathcal{U}}(X_{S_a}(t,z))^{3-D} \widehat{\mathcal{F}}(X_{S_a}(t,z))^{-\alpha}. \tag{46}$$

When $m = 0$, and hence $\widehat{\mathcal{F}}(X_{S_a}(t,z)) = t^2 z \, \phi_{S_a}(t,z)$, this same equality becomes

$$\begin{aligned} C_0(s; 0) = -\Gamma(1+\epsilon) \sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z \, \frac{t}{(1+z)^2} \, &\widehat{\mathcal{U}}(X_{S_a}(t,z))^{-1+2\epsilon} \\ &\times (t^2 z)^{-1-\epsilon} \, \phi_{S_a}(t,z)^{-1-\epsilon}. \end{aligned} \tag{47}$$

### 3.1 Using subtraction terms

The subtraction construction is the one used to expose the Laurent poles in $\epsilon$. At interior points one evaluates

$$\phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_s(y)}. \tag{48}$$

Boundary values are supplied by the continuous extension, using derivatives. For the first endpoint sector,

$$\phi_{S_1}(0,z) = \frac{1}{2z}\left[\partial_T^2 F_{S_1}(T,z)\right]_{T=0}, \tag{49}$$

$$\phi_{S_1}(t,0) = \frac{1}{t^2}\left[\partial_Z F_{S_1}(t,Z)\right]_{Z=0}, \tag{50}$$

$$\phi_{S_1}(0,0) = \frac{1}{2}\left[\partial_T^2 \partial_Z F_{S_1}(T,Z)\right]_{T=0,\,Z=0}. \tag{51}$$

Again, the derivatives of $F_{S_1}$ are not symbolic derivatives of an opened polynomial. They are derivatives of $\widehat{\mathcal{F}} \circ X_{S_1}$, obtained from the black-box derivative jets and the chain rule.

For $D = 4 - 2\epsilon$, write each factorized endpoint integrand as

$$I_{S_a}^{\mathrm{ind}}(t,z) = t^{-1-2\epsilon}\, z^{-1-\epsilon}\, G_{S_a}(t,z), \qquad a = 1,2, \tag{52}$$

where

$$G_{S_a}(t,z) = \frac{1}{(1+z)^2}\, U_{S_a}(t,z)^{-1+2\epsilon}\, \phi_{S_a}(t,z)^{-1-\epsilon}. \tag{53}$$

The two-dimensional endpoint subtraction identity is

$$\begin{aligned}
&\int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, t^{-1-2\epsilon} z^{-1-\epsilon} G(t,z) \\
&= \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, t^{-1-2\epsilon} z^{-1-\epsilon} \left[G(t,z) - G(0,z) - G(t,0) + G(0,0)\right] \\
&\quad - \frac{1}{2\epsilon} \int_0^1 \mathrm{d}z\, z^{-1-\epsilon} \left[G(0,z) - G(0,0)\right] \\
&\quad - \frac{1}{\epsilon} \int_0^1 \mathrm{d}t\, t^{-1-2\epsilon} \left[G(t,0) - G(0,0)\right] + \frac{G(0,0)}{2\epsilon^2}.
\end{aligned} \tag{54}$$

Each bracket is less singular at the corresponding endpoint. This is the mechanism that turns endpoint singularities into explicit poles and finite integrals. Applied sector by sector, it gives the full massless triangle as

$$\begin{aligned}
C_0(s;0) = -\Gamma(1+\epsilon) \sum_{a=1}^{2} \Bigg\{ &\int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, t^{-1-2\epsilon} z^{-1-\epsilon} \Big[G_{S_a}(t,z) - G_{S_a}(0,z) \\
&\qquad - G_{S_a}(t,0) + G_{S_a}(0,0)\Big] \\
&- \frac{1}{2\epsilon} \int_0^1 \mathrm{d}z\, z^{-1-\epsilon} \left[G_{S_a}(0,z) - G_{S_a}(0,0)\right] \\
&- \frac{1}{\epsilon} \int_0^1 \mathrm{d}t\, t^{-1-2\epsilon} \left[G_{S_a}(t,0) - G_{S_a}(0,0)\right] + \frac{G_{S_a}(0,0)}{2\epsilon^2} \Bigg\}.
\end{aligned} \tag{55}$$

For the explicit massless triangle in sector $S_1$,

$$U_{S_1}(t,z) = 1, \qquad \phi_{S_1}(t,z) = \frac{-s - \mathrm{i}0}{(1+z)^2}. \tag{56}$$

Thus

$$G^{\exp}_{S_1}(t,z) = (-s-\mathrm{i}0)^{-1-\epsilon}(1+z)^{2\epsilon}. \tag{57}$$

This function is independent of $t$. Therefore

$$G^{\exp}_{S_1}(t,z) - G^{\exp}_{S_1}(0,z) - G^{\exp}_{S_1}(t,0) + G^{\exp}_{S_1}(0,0) = 0, \tag{58}$$

and

$$G^{\exp}_{S_1}(t,0) - G^{\exp}_{S_1}(0,0) = 0. \tag{59}$$

The only non-trivial subtraction piece is

$$G^{\exp}_{S_1}(0,z) - G^{\exp}_{S_1}(0,0) = (-s-\mathrm{i}0)^{-1-\epsilon}\left[(1+z)^{2\epsilon}-1\right]. \tag{60}$$

The explicitly subtracted result for one endpoint sector is therefore

$$\begin{aligned}&\int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z\, I^{\mathrm{dir}}_{S_1}(t,z)\\ &= -\frac{(-s-\mathrm{i}0)^{-1-\epsilon}}{2\epsilon}\int_0^1 \mathrm{d}z\, z^{-1-\epsilon}\left[(1+z)^{2\epsilon}-1\right] + \frac{(-s-\mathrm{i}0)^{-1-\epsilon}}{2\epsilon^2}.\end{aligned} \tag{61}$$

The second sector gives the same contribution, so the full massless triangle is

$$C_0(s;0) = -2\Gamma(1+\epsilon)\left\{-\frac{(-s-\mathrm{i}0)^{-1-\epsilon}}{2\epsilon}\int_0^1 \mathrm{d}z\, z^{-1-\epsilon}\left[(1+z)^{2\epsilon}-1\right] + \frac{(-s-\mathrm{i}0)^{-1-\epsilon}}{2\epsilon^2}\right\}. \tag{62}$$

This is still a compact distributional expression. The object implemented in `FSD` is the Laurent expansion of this expression. Equivalently, the endpoint weights are expanded as plus distributions,

$$y^{-1-a\epsilon} = -\frac{1}{a\epsilon}\,\delta(y) + \sum_{n=0}^{\infty}\frac{(-a\epsilon)^n}{n!}\left[\frac{\log^n y}{y}\right]_+, \qquad a>0, \tag{63}$$

or, after pairing with a smooth test function $h$,

$$\int_0^1 \mathrm{d}y\, y^{-1-a\epsilon}h(y) = -\frac{h(0)}{a\epsilon} + \sum_{n=0}^{\infty}\frac{(-a\epsilon)^n}{n!}\int_0^1 \mathrm{d}y\,\frac{\log^n y}{y}\left[h(y)-h(0)\right]. \tag{64}$$

The subtraction formula above arranges precisely that the non-localized remainders have the required endpoint zeros, so the plus prescription is realized numerically as ordinary integrable functions with explicit logarithms. For the explicit $S_1$ remainder,

$$\begin{aligned}z^{-1-\epsilon}\left[(1+z)^{2\epsilon}-1\right] &= \epsilon\,\frac{2\log(1+z)}{z}\\ &\quad + \epsilon^2\,\frac{2\log^2(1+z) - 2\log z\,\log(1+z)}{z} + \mathcal{O}(\epsilon^3),\end{aligned} \tag{65}$$

which is regular at $z=0$. The same expansion is applied to the prefactor $(-s-\mathrm{i}0)^{-1-\epsilon}$, to $\Gamma(1+\epsilon)$, and to the black-box coefficient $G_{S_a}$ in the generic implementation. Thus `FSD` samples the coefficients of the Laurent series, not the unexpanded $z^{-1-\epsilon}$ distribution itself. This expression was obtained without requiring any symbolic expansion in an implementation: only $X_{S_a}$, $J_{S_a}$, the known monomial $t^2 z$, and black-box values and derivative limits of $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ are required in each endpoint sector.

## 4 General parametric structure

For a scalar $L$-loop integral with $N$ propagators of powers $\nu_i$, define

$$A = \sum_{i=1}^{N} \nu_i, \qquad D = D_0 + D_\epsilon \epsilon. \tag{66}$$

With the usual projective Feynman-parameter normalization, the parametric integral has the schematic form

$$I(\nu) = \mathcal{P}(D,\nu)\, \frac{\Gamma(A - LD/2)}{\prod_i \Gamma(\nu_i)} \int_{x_i \geq 0} \left[\prod_{i=1}^{N} \mathrm{d}x_i\, x_i^{\nu_i - 1}\right] \delta\left(1 - \sum_i x_i\right) \times \mathcal{U}(x)^{A-(L+1)D/2}\, \mathcal{F}(x)^{-(A-LD/2)}. \tag{67}$$

The convention-dependent prefactor $\mathcal{P}(D,\nu)$ contains the signs, powers of $\mathrm{i}\pi^{D/2}$, and any global normalization choices. It is not part of the sector-local black-box construction. What the sector processor needs are the affine powers

$$p_U(\epsilon) = A - (L+1)D/2, \tag{68}$$

$$p_F(\epsilon) = -(A - LD/2), \tag{69}$$

the parameter weights $\nu_i - 1$, and the retained black-box evaluators for $\mathcal{U}$ and $\mathcal{F}$.

After a sector map $x = X_s(y)$, the topology-specific sector generator should separate each possible endpoint monomial source,

$$J_s(y) \prod_i X_{s,i}(y)^{\nu_i - 1} N_s(y) = J_{s,\mathrm{reg}}(y) \prod_a y_a^{j_{s,a}}, \tag{70}$$

$$\mathcal{U}(X_s(y)) = M_{U,s}(y)\, \psi_s(y), \qquad M_{U,s}(y) = \prod_a y_a^{u_{s,a}}, \tag{71}$$

$$\mathcal{F}(X_s(y)) = M_{F,s}(y)\, \phi_s(y), \qquad M_{F,s}(y) = \prod_a y_a^{f_{s,a}}. \tag{72}$$

Here $N_s$ denotes any smooth numerator or extra parameter-space factor. The regular functions $\psi_s$ and $\phi_s$ are evaluated by black-box dual-number limits of $\mathcal{U}$ and $\mathcal{F}$; neither polynomial is symbolically opened inside the processor. The full endpoint power of $y_a$ is then

$$\rho_{s,a}(\epsilon) = j_{s,a} + u_{s,a}\, p_U(\epsilon) + f_{s,a}\, p_F(\epsilon), \tag{73}$$

with any additional numerator or measure monomial powers included in $j_{s,a}$. This is the object that decides the subtraction problem. The logarithmic endpoint case is the simplest one,

$$\rho_{s,a}(\epsilon) = -1 + c_{s,a}\epsilon, \qquad c_{s,a} \neq 0, \tag{74}$$

which produce explicit Laurent poles from $\int_0^1 \mathrm{d}y_a\, y_a^{-1+c_{s,a}\epsilon}$. The generic interface is not restricted to this case: if $\rho_{s,a}(\epsilon) = \beta_{s,a} + c_{s,a}\epsilon$ with a negative integer $\beta_{s,a}$, the sector data must request the Taylor projector through order $-\beta_{s,a} - 1$.

For a uniform notation, let

$$P \in \{\mathcal{U}, \mathcal{F}\}, \qquad r_{P,s,a} \in \mathbb{N}, \qquad M_{P,s}(y) = \prod_{a \in A_s} y_a^{r_{P,s,a}}, \tag{75}$$

with $r_{\mathcal{U},s,a} = u_{s,a}$, $r_{\mathcal{F},s,a} = f_{s,a}$, and $A_s$ the set of sector axes on which an endpoint monomial has been declared. Define

$$P_s(y) = \widehat{P}(X_s(y)), \qquad R_{P,s}(y) = \frac{P_s(y)}{M_{P,s}(y)}. \tag{76}$$

Thus $R_{\mathcal{U},s} = \psi_s$ and $R_{\mathcal{F},s} = \phi_s$. At an interior point this is just the quotient above. At a boundary point, let

$$B(y) = \{a \in A_s \mid y_a = 0\}, \qquad \bar{B} = A_s \setminus B, \tag{77}$$

and introduce dual variables $\eta_b$ for $b \in B$. The boundary jet is formed by

$$\widehat{y}_a(\eta) = \begin{cases} \eta_a, & a \in B, \\ y_a, & a \notin B. \end{cases} \tag{78}$$

Then the regular residual is the Taylor coefficient

$$R_{P,s}(y) = \frac{\left[\prod_{b\in B} \eta_b^{r_{P,s,b}}\right]\ \widehat{P}(X_s(\widehat{y}(\eta)))}{\prod_{a\in\bar{B}} y_a^{r_{P,s,a}}}. \tag{79}$$

Here $[\eta^n]$ denotes the Taylor coefficient, not the derivative. In derivative notation the same statement is

$$R_{P,s}(y) = \frac{\left(\prod_{b\in B} \frac{1}{r_{P,s,b}!}\, \partial_{\eta_b}^{r_{P,s,b}}\right) \widehat{P}(X_s(\widehat{y}(\eta)))\Big|_{\eta=0}}{\prod_{a\in\bar{B}} y_a^{r_{P,s,a}}}. \tag{80}$$

The chain rule is applied at the level of truncated jets:

$$J_B^{\mathbf{r}}(P_s)\,(y) = J_x^{\mathbf{r}}\Big(\widehat{P}\Big)(X_s(y)) \circ J_B^{\mathbf{r}}(X_s)\,(y), \tag{81}$$

where

$$\mathbf{r} = (r_{P,s,b})_{b\in B}. \tag{82}$$

Equivalently, for the first two derivative orders,

$$\partial_a P_s(y) = \sum_i \frac{\partial \widehat{P}}{\partial x_i}(X_s(y))\, \partial_a X_{s,i}(y), \tag{83}$$

$$\begin{aligned} \partial_a\partial_b P_s(y) = &\sum_{i,j} \frac{\partial^2 \widehat{P}}{\partial x_i \partial x_j}(X_s(y))\, \partial_a X_{s,i}(y)\, \partial_b X_{s,j}(y) \\ &+ \sum_i \frac{\partial \widehat{P}}{\partial x_i}(X_s(y))\, \partial_a\partial_b X_{s,i}(y). \end{aligned} \tag{84}$$

After extracting the declared monomials, every sector has the form

$$I_s(\epsilon) = \int_{[0,1]^d} \mathrm{d}^d y\, \left[\prod_{a\in A_s} y_a^{\rho_{s,a}(\epsilon)}\right] G_s(y,\epsilon), \tag{85}$$

where

$$G_s(y,\epsilon) = J_{s,\mathrm{reg}}(y)\, R_{\mathcal{U},s}(y)^{p_U(\epsilon)}\, R_{\mathcal{F},s}(y)^{p_F(\epsilon)} \tag{86}$$

times any remaining regular numerator factor. Write

$$\rho_{s,a}(\epsilon) = \beta_{s,a} + c_{s,a}\epsilon, \qquad \beta_{s,a} \in \mathbb{Z}, \tag{87}$$

and define the endpoint Taylor order

$$N_{s,a} = \max(-\beta_{s,a} - 1, 0). \tag{88}$$

For one endpoint variable,

$$\int_0^1 \mathrm{d}y\, y^{\beta+c\epsilon} G(y,\epsilon) = \int_0^1 \mathrm{d}y\, y^{\beta+c\epsilon} \left[ G(y,\epsilon) - \sum_{n=0}^{N} \frac{y^n}{n!} \partial_y^n G(0,\epsilon) \right] + \sum_{n=0}^{N} \frac{\partial_y^n G(0,\epsilon)}{n!\,(\beta+n+1+c\epsilon)}. \tag{89}$$

The first line is finite at $y = 0$, while the second line contains the explicit $\epsilon$-poles.

For several endpoint axes, introduce the Taylor projector

$$(\mathcal{T}_a G)(y,\epsilon) = \sum_{n=0}^{N_{s,a}} \frac{y_a^n}{n!} \left[ \partial_{y_a}^n G(y,\epsilon) \right]_{y_a=0}, \qquad \mathcal{R}_a = 1 - \mathcal{T}_a. \tag{90}$$

Then the recursive subtraction identity can be written compactly as

$$I_s(\epsilon) = \sum_{B\subseteq A_s} \sum_{\{0\le n_b\le N_{s,b}\}_{b\in B}} \left[ \prod_{b\in B} \frac{1}{n_b!\,(\beta_{s,b}+n_b+1+c_{s,b}\epsilon)} \right] \times \int \mathrm{d}y_{\bar{B}} \left[ \prod_{a\in A_s\setminus B} y_a^{\beta_{s,a}+c_{s,a}\epsilon} \right] \left[ \prod_{a\in A_s\setminus B} \mathcal{R}_a \right] \left[ \prod_{b\in B} \partial_{y_b}^{n_b} G_s(y,\epsilon) \Big|_{y_B=0} \right]. \tag{91}$$

In the logarithmic case $\beta_{s,a} = -1$, one has $N_{s,a} = 0$, and the identity reduces to

$$I_s(\epsilon) = \sum_{B\subseteq A_s} \left[ \prod_{b\in B} \frac{1}{c_{s,b}\epsilon} \right] \int \mathrm{d}y_{\bar{B}} \left[ \prod_{a\in A_s\setminus B} y_a^{-1+c_{s,a}\epsilon} \right] \times \left[ \prod_{a\in A_s\setminus B} (1-\mathcal{T}_a) \right] \left[ G_s(y,\epsilon) \right]_{y_B=0}, \qquad (\mathcal{T}_a G)(y,\epsilon) = G(y,\epsilon)|_{y_a=0}. \tag{92}$$

Finally, expanding

$$G_s(y,\epsilon) = \sum_{k\ge 0} \epsilon^k G_s^{(k)}(y), \qquad \prod_{a\in A_s\setminus B} y_a^{c_{s,a}\epsilon} = \sum_{r\ge 0} \frac{\epsilon^r}{r!} \left( \sum_{a\in A_s\setminus B} c_{s,a} \log y_a \right)^r, \tag{93}$$

gives the Laurent coefficients explicitly as ordinary finite integrals.

This audit leads to a useful interface boundary. A topology definition must retain the global parametric metadata: loop count, propagator powers, dimension, prefactor description, and affine powers of $\mathcal{U}$ and $\mathcal{F}$. A sector definition must retain the map, regular sector prefactor, monomial powers of $\mathcal{U}$ and $\mathcal{F}$, monomial powers from the Jacobian/measure or numerator, and the singular axes. The generic processor can then assemble $\rho_{s,a}$, evaluate $\psi_s$ and $\phi_s$ from dualized black-box evaluators, and apply the appropriate localized subtraction formula.

The numerical implementation now follows this interface boundary directly. The Laurent accumulator is variable length: once sectors are known, the minimum Laurent order is set by the scalar bound $-2L$, where $L$ is the loop count, and the requested maximum order is selected at run time. The sector endpoint pole depth is checked against this bound before integration. The subtraction engine is recursive for any number of endpoint axes with negative-integer base powers, using the Taylor order $N_{s,a} = -\beta_{s,a} - 1$. Logarithmic plus distributions are therefore the special case $N_{s,a} = 0$. Positive monomial powers factored by pySecDec but not belonging to singular axes are kept as regular multiplicative factors in $G_s$. Non-integer endpoint powers, unregulated endpoint powers, non-unit propagator powers, and tensor numerators are deliberately rejected in the current scalar implementation, because they require additional metadata or analytic input.

# 5 Outlook: Schwinger sampling with momentum-space numerators

The construction above assumes that the loop momenta have already been integrated out, so all non-trivial information enters through the Symanzik polynomials and possible parameter-space numerator factors. A complementary path is useful when the denominator part should still be organized in Feynman/Schwinger parameters, but the numerator is easier to keep as a momentum-space black box. Schematically, start from a regulated integral

$$I[f] = \int \mathrm{d}^{DL}k \, \frac{f(k)}{\prod_{e\in E}(D_e(k)+\mathrm{i}0)^{\nu_e}}. \tag{94}$$

Using Schwinger parameters, or equivalently the projective Feynman-parameter form of the same denominator product,

$$\prod_{e\in E}\frac{1}{(D_e(k)+\mathrm{i}0)^{\nu_e}} = \frac{\Gamma(\nu)}{\prod_e \Gamma(\nu_e)}\int_{\mathbb{P}^{|E|-1}_+}\prod_e \mathrm{d}x_e\, x_e^{\nu_e-1}\left[\sum_e x_e(D_e(k)+\mathrm{i}0)\right]^{-\nu}, \qquad \nu=\sum_e \nu_e, \tag{95}$$

one obtains an integral over positive edge parameters and loop momenta, without replacing $f(k)$ by tensor-reduction formulae. The quadratic denominator combination can be written schematically as

$$\sum_e x_e D_e(k) = (k-Q(x))^T A(x)(k-Q(x)) + \mathcal{V}(x), \qquad \mathcal{V}(x)=\frac{\mathcal{F}(x)}{\mathcal{U}(x)}, \tag{96}$$

with the appropriate $+\mathrm{i}0$ prescription or later contour deformation understood. Introducing an overall Schwinger scale and auxiliary momentum variables then gives a representation of the schematic form

$$\begin{aligned} I[f] \propto & \int_{\mathbb{P}^{|E|-1}_+}\prod_e \mathrm{d}x_e\, x_e^{\nu_e-1}\,\frac{1}{\mathcal{U}(x)^{D/2}\,\mathcal{V}(x)^{\omega}} \\ & \times \int_0^\infty \mathrm{d}\lambda\,\lambda^{\omega-1}e^{-\lambda}\int \mathrm{d}\mu(q)\, f(k(x,q,\lambda))\,, \end{aligned} \tag{97}$$

where

$$\omega = \sum_e \nu_e - \frac{DL}{2}. \tag{98}$$

Here $\mathrm{d}\mu(q)$ denotes a fixed reference measure for the auxiliary momentum variables after the quadratic form has been normalized. The important feature is not the particular choice of that reference measure. The important feature is that the numerator remains a callable function of momenta, while the Feynman-parameter part still exposes $\mathcal{U}$, $\mathcal{F}$, endpoint monomials, and sector maps to the same FSD machinery as before.

After sector decomposition of the parameter variables, the intended structure is

$$I[f] = \sum_s \int_{[0,1]^{d_s}} \mathrm{d}y \int \mathrm{d}\lambda\,\mathrm{d}\mu(q)\,\prod_{a\in A_s} y_a^{\rho_{s,a}(\epsilon)}\, G_s(y,\lambda,q;\epsilon)\, f(k_s(y,\lambda,q))\,. \tag{99}$$

The subtraction operators act on the sector variables $y_a$. The auxiliary variables $\lambda$ and $q$ are spectators for the endpoint algebra:

$$\mathcal{S}_s\left[G_s(y,\lambda,q;\epsilon)\, f(k_s(y,\lambda,q))\right], \tag{100}$$

with $\mathcal{S}_s$ the same local Taylor-subtraction operator used for the ordinary parameter-space integrand. Thus the future topology object would store not only $\mathcal{U}$, $\mathcal{F}$, and the parametric

exponents, but also the momentum routing, the quadratic denominator data, and the map from sampled auxiliary variables to loop momenta. The integrand definition would expose a momentum-space numerator callback

$$f:\ k \longmapsto f(k), \tag{101}$$

and FSD would provide

$$z \in [0,1]^n \longmapsto \left(K_{\mathrm{FSD}}(z),\, k(z)\right), \tag{102}$$

so that the numerical integrand is $K_{\mathrm{FSD}}(z) f(k(z))$.

This is conceptually different from momentum-space tropical sampling, such as the approach implemented in `momtrop`; see `https://github.com/alphal00p/momtrop`. The FSD version would deliberately use sector decomposition and subtraction for the Feynman-parameter part. It therefore inherits the larger sector count and will not compete with tropical sampling on the factorial-sector problem for finite parameter integrals. The advantage is different: the same framework can expose endpoint poles in $\epsilon$, and, once contour deformation or another threshold treatment is added, it is not tied to a finite Euclidean integration domain. For generic higher-order endpoint subtractions one may also need derivatives of the composed numerator $f(k_s(y,\lambda,q))$, or an explicit regularity contract showing that only boundary values are required. This is a boundary condition for the future backend, not part of the current scalar implementation. Moreover, the momentum-space numerator must be understood as a $D$-dimensional numerator, not merely a four-dimensional one. Rational terms from the $(D-4)$-dimensional components have to be retained, for example through the usual $\mu$-term bookkeeping

$$k^2_{[D]} = k^2_{[4]} + \mu^2, \qquad \mu^2 \equiv k^2_{[D-4]}, \tag{103}$$

with signs and metric conventions fixed consistently with the chosen denominator prescription.

# 6 Implementation in `FSD`

The repository contains the modular implementation used for the numerical checks described above. It is referred to simply as `FSD`. The executable entry point is

$$\texttt{FSD.py}. \tag{104}$$

A local virtual environment is created at

$$\texttt{.venv}, \tag{105}$$

and OneLOopBridge is required as an external dependency. The benchmark values are obtained from the Python binding calls

$$\texttt{oneloop\_bridge.three\_point(0,0,s,m2,m2,m2)}, \tag{106}$$

$$\texttt{oneloop\_bridge.four\_point(0,0,0,0,s12,s23,m2,m2,m2,m2)}. \tag{107}$$

The only implementation described here is the modular `FSD` code.

## 6.1 Code organization

The implementation is organized so that the black-box rule is visible directly in the code layout:

- `FSD.py` is the command-line steering layer. It parses options, validates kinematics, builds the request object, prints summaries, calls the benchmark, launches the integration, and renders the final table.

- `sectors_generator.py` owns the declarative sector data: sector maps $X_s$, regular sector prefactors, endpoint monomials $M_{U,s}$ and $M_{F,s}$, monomial powers from Jacobian/measure or numerator factors, and singular axes. The lowering of these declarative expressions to Symbolica callbacks is an explicit preparation step.
- `integrand.py` owns the topology definition and the generic `SectorProcessor`. The topology retains $\mathcal{U}$ and $\mathcal{F}$ for display and evaluator construction, but the processor never substitutes a sector map into them symbolically.
- `subtraction_formula.py` owns the generated Symbolica endpoint-subtraction formulae. It provides both the full sector-signature formula backend and the newer lower-signature endpoint-projector backend.
- `integrator.py` owns the Havana sampling loop, manual Laurent coefficient accumulation, progress reporting, target-accuracy stopping, and hot-path timing profile.
- `benchmark.py`, `formatting.py`, `definitions.py`, and `utils.py` isolate OneLOopBridge access, colored output, shared data containers, and parenthesis uncertainty formatting.
- `dot_parser.py`, `kinematics.py`, `dot_topology.py`, and `pysecdec_bridge.py` implement the experimental GammaLoop-convention DOT-file path. The bridge is the only FSD-owned module that imports pySecDec. It uses pySecDec to construct $\mathcal{U}, \mathcal{F}$ and sector data, then converts them into the same `TopologyDefinition` and `SectorDefinition` objects consumed by the generic processor.

### 6.2 DOT-file path through pySecDec

The command-line option

```
--dot-file path/to/integral.dot
```

selects the GammaLoop DOT-backed topology path, together with

```
--kinematics path/to/kinematics.yaml
```

The YAML file supplies numeric symbol values and scalar-product replacement rules. The current schema is

```
values:
  s12: -1.0
  s23: -1.0
  mt: 0.0
replacements:
  p1*p1: 0
  p2*p2: 0
  p1*p2: s12/2
```

Values and replacement right-hand sides are evaluated with Symbolica, not with SymPy. DOT edges touching nodes with `style=invis` are external half-edges. An edge `ext -> v` is interpreted as incoming momentum, and `v -> ext` as outgoing momentum. Internal edge attributes `mass` are either numeric strings or symbols present in the YAML `values` block. These mass symbols are resolved to their numeric values before pySecDec generation, so the decomposition sees the actual massless or massive polynomial for the requested kinematic point. Flow directions are retained in the parsed DOT data; pySecDec receives bare external momentum symbols, with the sign convention represented by the scalar-product replacement rules.

The implemented generation sequence is

$$\texttt{DOT file} \longrightarrow \texttt{ParsedDotGraph}, \tag{108}$$

$$\texttt{ParsedDotGraph} + \texttt{KinematicsDefinition} \longrightarrow \texttt{pySecDec LoopIntegralFromGraph}, \tag{109}$$

$$\texttt{pySecDec LoopIntegralFromGraph} \longrightarrow \texttt{TopologyDefinition}, \tag{110}$$

$$\texttt{pySecDec sectors} \longrightarrow \{\texttt{SectorDefinition}\}. \tag{111}$$

The selected decomposition method is controlled by

```
--sector-method geometric
--sector-method geometric_ku
--sector-method iterative
```

The default is `iterative`, which does not require Normaliz. For geometric decomposition, Normaliz is required either from pySecDecContrib or through `--normaliz-executable`. If no Normaliz executable is visible, FSD rejects the geometric mode before generation starts and the user can select `geometric_ku` or `iterative` instead. The execution engine is controlled by

```
--dot-engine fsd
--dot-engine pysecdec
--dot-engine both
```

In `fsd` mode, pySecDec is used only for generation; integration is performed by FSD/Havana using black-box Symbolica evaluators. In `pysecdec` mode, pySecDec generates, compiles, loads, and runs its own C++ integration package. In `both` mode both paths are run for comparison.

Generation timings are recorded at detailed granularity and also grouped into three headline buckets:

$$T_{\mathcal{U},\mathcal{F}} := T_{\mathrm{DOT}} + T_{\mathrm{kin}} + T_{\mathrm{LoopIntegralFromGraph}} + T_{\mathrm{U/F\ extraction}}, \tag{112}$$

$$T_{\mathrm{sectors}} := T_{\mathrm{pySecDec\ decomposition}} + T_{\mathrm{SectorDefinition\ conversion}}, \tag{113}$$

$$T_{\mathrm{Symbolica}} := T_{\mathrm{scalar\ U/F\ evaluators}} + T_{\mathrm{sector\ map/J\ evaluators}} + T_{\mathrm{dual\ U/F\ evaluators}}. \tag{114}$$

The pre-integration summary prints these three buckets, $\mathcal{U}, \mathcal{F}$, evaluator parameter order, the dual-evaluator mode, a capped sector table showing at most twenty sectors, and statistics for total sector count, number of singular axes, U/F monomial power tuples, endpoint pole depth, maximum dimension, and maximum Laurent pole order.

The current DOT examples include the massless triangle and box, plus six massive Euclidean multi-loop topologies: a two-loop two-point kite, a three-loop two-point self-energy graph, two two-loop three-point graphs, and two three-loop three-point graphs. The larger three-point examples have six and eight internal lines, respectively. The triangle and box reproduce the hard-coded examples through the DOT path. The multi-loop examples are finite in the tested kinematics and are run with pySecDec contour deformation disabled. They exercise pySecDec U/F construction, sector conversion, retained Symbolica evaluators, regular monomial factors, and the same FSD/Havana integration loop without the package-generation cost of double-box ladder examples. DOT examples with global Gamma prefactor poles are not yet part of the validated set, because the current pySecDec-convention prefactor convolution assumes a prefactor regular at $\epsilon = 0$.

### 6.3 Prepared sectors and black-box processing

The code mirrors the split used in this note. Sectors are prepared independently of the Symanzik polynomials. Each sector stores only

$$\{X_s,\ J_{s,\mathrm{reg}},\ M_{U,s},\ M_{F,s},\ \text{measure/numerator monomial powers, singular axes}\}, \tag{115}$$

plus Symbolica evaluators generated from these known sector expressions. The processing layer receives these prepared sectors together with black-box Symbolica evaluators

$$\widehat{\mathcal{U}}(x;p), \qquad \widehat{\mathcal{F}}(x;p), \tag{116}$$

where $p$ denotes the external invariants and masses. After the evaluators are built, the processing code does not inspect or symbolically rewrite the expressions for $\mathcal{U}$ or $\mathcal{F}$.

At runtime the processor receives only a `SectorDefinition`, a `TopologyDefinition`, and numeric sample points. For a sector point $y$ it evaluates

$$x = X_s(y), \qquad \widehat{\mathcal{U}}(x), \qquad \widehat{\mathcal{F}}(x), \qquad J_s(y), \tag{117}$$

through prebuilt Symbolica evaluators. If the sector has no endpoint monomial, the contribution is evaluated directly. If a monomial is declared, the residuals

$$\psi_s(y) = \frac{\widehat{\mathcal{U}}(X_s(y))}{M_{U,s}(y)}, \qquad \phi_s(y) = \frac{\widehat{\mathcal{F}}(X_s(y))}{M_{F,s}(y)} \tag{118}$$

are evaluated by these quotients in the interior. On endpoint samples the same quantities are recovered as Taylor coefficients of the composed black-box $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$. In the default backend these coefficients are obtained from dualized $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators. The dual jets of the known sector map provide the chain-rule input, so no symbolic expansion of $\mathcal{U}(X_s(y))$ or $\mathcal{F}(X_s(y))$ is performed. The scalar $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators are constructed once with the topology. The command line exposes the following Taylor-evaluator preparation policies:

- `--pregenerate-dual-evaluators`, the default, builds one dualized $\widehat{\mathcal{U}}$ evaluator and one dualized $\widehat{\mathcal{F}}$ evaluator for each unique sector Taylor shape before integration starts.
- `--lazy-dual-evaluators-generation` keeps the cache-on-first use behavior: the scalar evaluator is copied and dualized the first time a shape is requested. This time is reported separately as Taylor evaluator setup time, not as ordinary evaluator time.
- `--pregenerate-single-overall-dual-evaluator` builds an envelope dual evaluator. For an integration dimension $d$, let $r_d = \max_s |A_s|$ be the maximum number of singular axes among sectors of that dimension. Each sector multi-index $\alpha \in \mathbb{N}^{|A_s|}$ is embedded as
$$\iota_s(\alpha) = (\alpha_1, \ldots, \alpha_{|A_s|}, 0, \ldots, 0) \in \mathbb{N}^{r_d}. \tag{119}$$
The envelope shape is
$$\mathcal{J}_d^{\text{env}} = \bigcup_{s:\dim s=d} \{\iota_s(\alpha) \,:\, \alpha \in \mathcal{J}_s\}. \tag{120}$$
After evaluation, the output columns indexed by $\iota_s(\alpha)$ are remapped back to the sector-native shape $\mathcal{J}_s$.
- `--symbolic-derivatives` does not dualize the topology-level $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{F}}$ evaluators. Instead, it builds ordinary non-dual Symbolica evaluators for the symbolic partial derivatives
$$\partial_x^\rho \widehat{P}(x), \qquad \widehat{P} \in \{\widehat{\mathcal{U}}, \widehat{\mathcal{F}}\}, \tag{121}$$
with $\rho$ selected from the polynomial support and the largest sector Taylor order requested. These derivative evaluators are shared by all sectors. For a sector map
$$X_s(y+\eta) = X_s(y) + H_s(y,\eta), \tag{122}$$
the sector Taylor coefficients are reconstructed numerically through
$$[\eta^\alpha]\, \widehat{P}(X_s(y+\eta)) = [\eta^\alpha] \sum_\rho \frac{\partial_x^\rho \widehat{P}(X_s(y))}{\rho!}\, H_s(y,\eta)^\rho. \tag{123}$$

This is an explicit chain-rule backend for cross-checking the dualized evaluator path; it is not expected to be faster in the current Python implementation.

In FSD DOT mode, `TopologyDefinition` and `SectorDefinition` objects are generated and prepared once in the master process. Multi-worker integration uses a fork context so workers inherit this prepared state and wrap it in a `SectorProcessor`. Workers do not call pySecDec, do not rerun sector generation, and do not create Normaliz temporary directories. If a fork context is unavailable, multi-worker DOT integration fails explicitly instead of silently regenerating sectors at runtime.

The generic processor then constructs

$$g_s(y;\epsilon) = J_{s,\mathrm{reg}}(y)\,\psi_s(y)^{p_U(\epsilon)}\,\phi_s(y)^{p_F(\epsilon)}, \tag{124}$$

multiplied by any regular positive monomial factors from the sector metadata. It then assembles the endpoint powers $\rho_{s,a}(\epsilon)$ and applies the recursive localized Taylor-subtraction formula for all declared singular axes. Higher-rank endpoint structures therefore use the same code path as the one- and two-axis triangle/box examples. Unsupported endpoint structures, such as non-integer powers or powers without an epsilon regulator, raise an explicit diagnostic instead of being guessed.

### 6.4 Subtraction backends and lower-signature projector

The command-line option

```
--subtraction-backend recursive|formula|projector-formula
```

selects how the endpoint algebra is evaluated after the sector data and U/F coefficient evaluators have been prepared.

The `recursive` backend is the direct vectorized Python/Numpy implementation of Eq. (91). It obtains the regular Taylor–Laurent coefficients

$$g^{(Z)}_{s,\alpha,r}(y_{\bar{Z}}) = [\epsilon^r \eta^\alpha]\, G_s(y_{\bar{Z}}, \eta_Z; \epsilon) \tag{125}$$

from the black-box U/F/J Taylor evaluators, and then performs the inclusion–exclusion sum over endpoint projectors in Python.

The `formula` backend builds one Symbolica evaluator for the complete sector-subtraction expression. Its cache signature contains the sector axes, all U/F/Jacobian/measure monomial powers, the native Taylor-coefficient layout, the endpoint powers, and the requested Laurent range. This path is a useful correctness reference because the endpoint algebra is evaluated inside Symbolica, but the formula is close to sector-specific.

The `projector-formula` backend splits the expression into two layers. First, the already-existing black-box Taylor path computes the sector-specific regular coefficients $g^{(Z)}_{s,\alpha,r}$. Second, these coefficients are passed to a Symbolica evaluator that knows only the endpoint projector algebra,

$$\mathcal{P}_{\{\beta_a,c_a,N_a\},\,k_{\min}:k_{\max}} : \left\{ y_a,\ g^{(Z)}_{s,\alpha,r} \right\} \longmapsto \{I_{s,k}(y)\}_{k=k_{\min}}^{k_{\max}}. \tag{126}$$

Its cache key is only

$$(n_{\mathrm{axes}}, \{(\beta_a, c_a)\}_{a=1}^{n_{\mathrm{axes}}}, \{N_a\}_{a=1}^{n_{\mathrm{axes}}}, \{k_{\min}, \ldots, k_{\max}\}), \tag{127}$$

not the sector map or the U/F polynomial data. Thus U and F remain black-box coefficient providers, while many sectors can share the same generated endpoint-projector evaluator.

For the DOT double-box test this reduces the generated subtraction expression family from 133 full sector formula signatures to 20 endpoint-projector signatures. For the stored triple-box metadata, the corresponding count is about 863 current full signatures versus 158 ordered endpoint-projector signatures. The present `projector-formula` backend is therefore a lower-signature generated-formula path. It does not yet remove all Python cost, because the regular coefficients $g^{(Z)}_{s,\alpha,r}$ are still assembled by the shared Python/Numpy Taylor-series layer.

### 6.5 Havana sampling and stopping

`FSD` uses Symbolica's Havana integrator as a sampler rather than as a single black-box scalar integrator. One discrete dimension chooses the sector and each sector has a continuous adaptive grid of the same dimension:

$$\text{discrete sector id} \quad \otimes \quad y \in [0,1]^d. \tag{128}$$

Lower-support subtraction pieces are localized into the same highest-dimensional sample space by dummy unit-density dependence on unused variables. This avoids separate grids for bulk, edge, and corner pieces.

Each sampled point contributes to all Laurent coefficients through the same manual accumulator,

$$A_k = \frac{1}{N}\sum_{n=1}^{N} w_n \, I_{k,s_n}(y_n), \qquad k \in \{k_{\min}, k_{\min}+1, \ldots, k_{\max}\}. \tag{129}$$

Here $k_{\min} = -2L$ by default, and the upper endpoint of the range is the CLI-selected maximum epsilon order. Havana is trained on the norm of the last requested Laurent coefficient, so the training observable follows the coefficient actually controlling the requested truncation. The adaptive grid is cloned at the beginning of an iteration; batch training samples are accumulated into the clone and merged back into the live grid only at the end of a completed iteration.

The target-accuracy stop is evaluated after each accumulated batch, not only after a full Havana iteration. Thus the final returned coefficients contain all completed batches up to the stopping point. In parallel mode the driver keeps only a worker-sized window of batches in flight, so early stopping does not submit the entire iteration unnecessarily. The option `--batch-size` therefore controls both vectorization size and mid-iteration stopping granularity.

### 6.6 Timing profile

The timing profile reported by the progress bar and final table is a work-time profile:

$$T_{\text{prof}} = T_{\text{Python}} + T_{\text{eval}} + T_{\text{Havana}}. \tag{130}$$

Here $T_{\text{eval}}$ is the sum of Symbolica evaluator wall times over all workers, $T_{\text{Python}}$ is NumPy and Python glue on workers plus master-side submission, result retrieval, accumulation, and progress rendering, and $T_{\text{Havana}}$ is Havana sampling, training accumulation, grid merge, and grid update time. The displayed evaluator average is

$$\langle t_{\text{eval}} \rangle = \frac{10^6 \, T_{\text{eval}}}{N_{\text{accepted}}} \quad \text{microseconds per sample per worker,} \tag{131}$$

because $T_{\text{eval}}$ is already summed over workers. Taylor evaluator setup is tracked separately as $T_{\text{Taylor gen}}$. For the default pregenerated dual mode and for the symbolic-derivative mode this belongs to the generation bucket $T_{\text{Symbolica}}$. For lazy mode it can occur during the first integration batches, but it is still reported separately and is not included in $T_{\text{eval}}$.

### 6.7 Implemented modes and conventions

For the triangle, the command-line interface supports massive and massless modes. In the validated finite massive region, endpoint monomial factorization is not applied. The implementation evaluates

$$C_0(s;m^2) = -\sum_{a=1}^{2} \int_0^1 \mathrm{d}t \int_0^1 \mathrm{d}z \, J_{S_a}(t,z) \, \widehat{\mathcal{U}}(X_{S_a}(t,z))^{-1} \, \widehat{\mathcal{F}}(X_{S_a}(t,z))^{-1}. \tag{132}$$

In massless Euclidean mode, the implementation uses the subtraction formula of Section 3.2 and returns the coefficients of

$$C_0(s;0) = \frac{C_{-2}}{\epsilon^2} + \frac{C_{-1}}{\epsilon} + C_0 + \mathcal{O}(\epsilon). \tag{133}$$

The default output convention is the OneLOopBridge convention. In this convention the universal Gamma/Euler factors are stripped from the displayed Laurent coefficients. For massless endpoint representations this also amounts to applying the finite convention shift described in the direct construction. The option `--prefactor-convention` controls only the overall scalar-integral normalization shown in the output table. The default value `raw` displays the raw OneLOopBridge normalization. The value `feynman` multiplies both the `FSD` coefficients and the benchmark coefficients by

$$\texttt{TO\_FEYNMAN} = -\frac{1}{16\pi^2}. \tag{134}$$

In either display convention, values with Monte Carlo uncertainty are printed with the two-significant-digit parenthesis notation.

### 6.8 Implemented box example

The implementation also contains a scalar box example with massless external legs,

$$D_0(0,0,0,0,s_{12},s_{23};m^2). \tag{135}$$

For equal internal masses, the simplex polynomial used by the implementation is

$$\mathcal{U} = x_0 + x_1 + x_2 + x_3, \qquad \mathcal{F} = m^2\mathcal{U}^2 - s_{12}x_0x_2 - s_{23}x_1x_3 - \mathrm{i}0. \tag{136}$$

After imposing $\mathcal{U} = 1$, the finite four-point integral in $D = 4$ is

$$D_0 = \sum_{b=0}^{3} \int_0^1 \mathrm{d}y_0 \int_0^1 \mathrm{d}y_1 \int_0^1 \mathrm{d}y_2\, J_b(y)\, \widehat{\mathcal{F}}(X_b(y))^{-2}. \tag{137}$$

Here $b$ labels the primary sector in which $x_b$ is the largest Feynman parameter. In sector $b$,

$$x_b = \frac{1}{1+y_0+y_1+y_2}, \qquad x_{j\neq b} = \frac{y_j}{1+y_0+y_1+y_2}, \tag{138}$$

with the three $y_j$'s assigned in increasing order to the non-dominant parameters. The Jacobian is

$$J_b(y) = \frac{1}{(1+y_0+y_1+y_2)^4}. \tag{139}$$

For $m = 0$, each primary sector has the form

$$\mathcal{F}(X_b(y)) = \frac{A_b\, y_\ell + B_b\, y_r y_s}{(1+y_0+y_1+y_2)^2}, \tag{140}$$

where one of the two bilinears $x_0x_2$ and $x_1x_3$ contains the dominant parameter and is therefore linear in the primary variables. The implementation does not need the symbolic values of $A_b$ and $B_b$; it only uses the known positions of the linear variable $y_\ell$ and product variables $y_r, y_s$. The three secondary sectors are

$$L: \quad y_r = uv, \quad y_\ell = u, \quad y_s = w, \qquad M_L = u, \tag{141}$$

$$P: \quad y_r = u, \quad y_s = v, \quad y_\ell = uvw, \qquad M_P = uv, \tag{142}$$

$$Q: \quad y_r = u, \quad y_\ell = uv, \quad y_s = vw, \qquad M_Q = uv. \tag{143}$$

The secondary Jacobian equals the displayed monomial in each case, so the regular prefactor after monomial extraction is again

$$\frac{1}{(1+y_0(u,v,w)+y_1(u,v,w)+y_2(u,v,w))^4}. \tag{144}$$

The residual coefficient

$$\phi_{b\alpha}(u,v,w)=\frac{\mathcal{F}(X_{b\alpha}(u,v,w))}{M_\alpha(u,v,w)} \tag{145}$$

is evaluated by the black-box $\mathcal{F}$ away from the endpoint and by dual-number Taylor coefficients at $u=0$ and $v=0$. The one-monomial sector $L$ uses the one-variable subtraction formula, while the $P$ and $Q$ sectors use the two-variable subtraction formula.

The raw coefficients produced directly by the massless box sectors are converted to the OneLOopBridge stripped convention as

$$D_{-2}=A_{-2},\qquad D_{-1}=A_{-1}+A_{-2},\qquad D_0=A_0+A_{-1}+\frac{\pi^2}{6}A_{-2}. \tag{146}$$

The current massless box command validates Euclidean kinematics, $s_{12}<0$ and $s_{23}<0$. Non-Euclidean massless kinematics are outside the present implementation.

### 6.9 Example commands

The massive triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s 1.0 --m 1.0
```

The same check in the Feynman-normalized prefactor convention can be displayed with

```
.venv/bin/python FSD.py \
  --integral triangle \
  --s 1.0 \
  --m 1.0 \
  --prefactor-convention feynman
```

The massless Euclidean triangle check can be run with

```
.venv/bin/python FSD.py --integral triangle --s -1.0 --m 0.0
```

The massive box example can be run with

```
.venv/bin/python FSD.py \
  --integral box \
  --s12 0.5 \
  --s23 0.7 \
  --m 1.0
```

The massless Euclidean box check can be run with

```
.venv/bin/python FSD.py \
  --integral box \
  --s12 -1.0 \
  --s23 -2.0 \
  --m 0.0
```

The same triangle and box can be routed through DOT input and pySecDec sector generation, for example

```
.venv/bin/python FSD.py \
  --dot-file examples/dot/triangle.dot \
  --kinematics examples/dot/triangle_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/box.dot \
  --kinematics examples/dot/box_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both
```

The compact multi-loop DOT smoke checks are

```
.venv/bin/python FSD.py \
  --dot-file examples/dot/kite_2loop.dot \
  --kinematics examples/dot/kite_2loop_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/self_energy_3loop.dot \
  --kinematics examples/dot/self_energy_3loop_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/three_point_2loop.dot \
  --kinematics examples/dot/three_point_2loop_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/three_point_3loop.dot \
  --kinematics examples/dot/three_point_3loop_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/three_point_2loop_6line.dot \
  --kinematics examples/dot/three_point_2loop_6line_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both

.venv/bin/python FSD.py \
  --dot-file examples/dot/three_point_3loop_8line.dot \
  --kinematics examples/dot/three_point_3loop_8line_kinematics.yaml \
  --sector-method iterative \
  --dot-engine both
```

Massless timelike commands, such as `--integral triangle --s 1.0 --m 0.0`, are rejected by validation. Built-in triangle and box runs use OneLOopBridge unless an explicit target is provided. DOT/FSD-only runs report no pull unless a target is provided or pySecDec is run through `--dot-engine both`. The implementation intentionally imports neither `scipy` nor `sympy`; all symbolic handling is done through Symbolica evaluators.